Fast matrix multiplication using CP decomposition

Computational laboratory project report

Alberto Defendi

Abstract

We develop a Rust crate (library) that given matrices $A, B \in \mathbb{R}^n$ computes their product $C = AB$ using fast matrix multiplication algorithms, for any given CP decomposition.

Contents

1 Notation and tensor preliminaries

The relevant notation for our work is in Table 1. We briefly review basic tensor preliminaries, following the notation in [1]. A *tensor* is a multidimensional array, and in this work we deal only with 3-dimensional (or 3-way) tensors $\mathcal{X} \in \mathbb{R}^{m \times n \times p}$. The *k-th frontal slice* of $\mathcal{X}$ is the matrix $\mathbf{X}_k$. For $\mathbf{u} \in \mathbb{R}^m$, $\mathbf{v} \in \mathbb{R}^n$, $\mathbf{w} \in \mathbb{R}^p$, we define the rank-1 *outer product* tensor $\mathcal{X} = \mathbf{u} \circ \mathbf{v} \circ \mathbf{w} \in \mathbb{R}^{m \times n \times p}$ with entries $x_{ijk} = u_i v_j w_k$. Addition of tensors is defined entry-wise. The *rank* of a tensor $\mathcal{X}$ is defined as the minimum number of rank-one tensors that generate $\mathcal{X}$ as their sum. We define the *CP* decomposition of rank R of a tensor $\mathcal{X}$ as the sum $\mathcal{X} = \sum_{r=1}^{R} u_r \circ v_r \circ w_r$. We denote it by $\mathcal{X} = [\![U, V, W]\!]$, where U, V and W are matrices with R columns given by u_r, v_r, and w_r. An important operation in the context of tensors is the *mode$-k$ product*, which is defined as. In this work, we will often use the equation $\mathcal{X} \times_1 a^\top \times_2 b^\top = c$, with $c_k = a^\top \mathbf{X}_{(k)} b = \sum_{i=1}^{m} \sum_{j=1}^{n} x_{ijk}$ to refer to matrix multiplication algorithms.

Table 1: Summary of notation.

$\langle M, K, N \rangle$	Bilinear form representing base case matrix multiplication of an $M \times K$ matrix by a $K \times N$ matrix.
$P \times Q \times R$	Dimensions of actual matrices that are multiplied ($P \times Q$ matrix multiplied by $Q \times R$ matrix).
$[\![\mathbf{U}, \mathbf{V}, \mathbf{W}]\!]$	Factor matrices corresponding to a tensor CP decomposition that provides a fast algorithm.
$\mathcal{X}$	Order-3 tensor.
$\mathcal{X} = \mathbf{u} \circ \mathbf{v} \circ \mathbf{w}$	Rank-1 tensor with entries $X_{ijk} = u_i v_j w_k$.
$\mathbf{X}_{(i)}$	ith frontal slice of $\mathcal{X}$, the matrix of entries $X_{:,:,i}$.
$\mathbf{a}_k$, A_{ij}	kth column and i, j entry of matrix $\mathbf{A}$.
$\text{vec}(\mathbf{A})$	Row-order vectorization of the entries of $\mathbf{A}$.
$\text{nnz}(\cdot)$	Number of non-zero entries of an object.

2 Fast matrix multiplication

Matrix multiplication is a bilinear operation. We indicate with $\langle m, n, p \rangle$ the bilinear form that represents the multiplication between a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $\mathbf{B} \in \mathbb{R}^{n \times p}$. Fixing the natural ordering on the entries of $\mathbf{A}$ and $\mathbf{B}$ and the inverse ordering on $\mathbf{C} = \mathbf{AB}$, we get a representation of $\mathcal{X} \in \mathbb{R}^{mn \times np \times mp}$ of the matrix multiplication $\langle m, n, p \rangle$ such that

explain

$$\text{vec}(\mathbf{C}^\top) = \mathcal{X} \times_1 \text{vec}(\mathbf{A})^\top \times_2 \text{vec}(\mathbf{B})^\top. \tag{1}$$

Note that we fix the linear order on A and B, while we fix the reverse ordering on C by applying the transpose operation.

why

If we are given a CP decomposition of rank r of the tensor $\mathcal{X} = [\![U, V, W]\!]$, we can rewrite Equation 1 into

Tensor definition

$$\text{vec}(\mathbf{C}^\top) = \sum_{l=1}^{r} (\underbrace{\mathbf{u_l}^\top \text{vec}(\mathbf{A})}_{s_l}) \cdot (\underbrace{\mathbf{v_l}^\top \text{vec}(\mathbf{B})}_{t_l}) \mathbf{w_l} = \sum_{l=1}^{r} (\underbrace{s_l \cdot t_l}_{m_l}) w_l, \tag{2}$$

where $\mathbf{u_l}$, $\mathbf{v_l}$, $\mathbf{w_l}$ denote the columns of $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$. This reduces the number of active multiplications to the rank r of $\mathcal{X}$.

2.1 The classic matrix multiplication algorithm tensor formulation

We present the $\langle 2, 2, 2 \rangle$ classical matrix multiplication algorithm in the context of tensors. When $m, n, p > 2$ we can always partition them in blocks as:

$$\begin{array}{c} \overbrace{}^{\lfloor n/2 \rfloor} \; \overbrace{}^{\lceil n/2 \rceil} \\ \lfloor m/2 \rfloor \left\{ \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \right. \end{array}, \quad \begin{array}{c} \overbrace{}^{\lfloor p/2 \rfloor} \; \overbrace{}^{\lceil p/2 \rceil} \\ \lfloor n/2 \rfloor \left\{ \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \right. \end{array}, \tag{3}$$

where the sizes of the subblocks are depicted in the above equation. This algorithmic formulation is

often called *divide et impera*, as it consists in breaking the problem into larger subproblems, and later recombining the results. In particular, we can view the product AB as the product between 2×2 block matrices

$$\text{(4)} \quad \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}.$$

The naive algorithm proceeds by combining a set of eight matrix multiplications with four additions:

$$\begin{aligned} M_1 &= A_{11} \cdot B_{11} & M_2 &= A_{12} \cdot B_{21} & M_3 &= A_{11} \cdot B_{12} \\ M_4 &= A_{12} \cdot B_{22} & M_5 &= A_{21} \cdot B_{11} & M_6 &= A_{22} \cdot B_{21} \\ M_7 &= A_{21} \cdot B_{12} & M_8 &= A_{22} \cdot B_{22} \end{aligned}$$

$$\begin{aligned} C_{11} &= M_1 + M_2 & C_{12} &= M_3 + M_4 \\ C_{21} &= M_5 + M_6 & C_{22} &= M_7 + M_8 \end{aligned}$$

For example, when $m = n = p = 2$, we get the tensor with frontal slices

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This yields, for example,

$$X_3 \times_1 \mathrm{vec}(A)^\top \times_2 \mathrm{vec}(B)^\top = \begin{bmatrix} a_{11} & a_{21} & a_{12} & a_{22} \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{21} \\ b_{12} \\ b_{22} \end{bmatrix} = a_{21}b_{11} + a_{22}b_{21} = c_{21}.$$

Note that the formula in Equation 2 in the context of block matrix multiplication in Equation 4 yelds to

$$\mathrm{vec}(A) = \begin{bmatrix} A_{11} \\ A_{21} \\ A_{12} \\ A_{22} \end{bmatrix}, \quad \mathrm{vec}(B) = \begin{bmatrix} B_{11} \\ B_{21} \\ B_{12} \\ B_{22} \end{bmatrix}, \quad \mathrm{vec}(C^\top) = \begin{bmatrix} C_{11} \\ C_{12} \\ C_{21} \\ C_{22} \end{bmatrix}.$$

Also the factors s_l and t_l in Equation 2 become

$$s_l = u_l^\top \mathrm{vec}(A) = \sum_{i=1}^{4} U_{li} \mathrm{vec}(A)_i, \quad t_l = V_l^\top \mathrm{vec}(B) = \sum_{i=1}^{4} V_{li} \mathrm{vec}(B)_i$$

2.2 Strassen algorithm

The most well known fast algorithm is due to Strassen, and follows the same block structure:

$$
\begin{aligned}
&S_1 = A_{11} + A_{22} && S_2 = A_{21} + A_{22} && S_3 = A_{11} \\
&S_4 = A_{22} && S_5 = A_{11} + A_{12} && S_6 = A_{21} - A_{11} \\
&S_7 = A_{12} - A_{22} \\
&T_1 = B_{11} + B_{22} && T_2 = B_{11} && T_3 = B_{12} - B_{22} \\
&T_4 = B_{21} - B_{11} && T_5 = B_{22} && T_6 = B_{11} + B_{12} \\
&T_7 = B_{21} + B_{22}
\end{aligned}
$$

$$
\begin{aligned}
&M_r = S_r T_r, \quad 1 \leq r \leq 7 \\
&C_{11} = M_1 + M_4 - M_5 + M_7 && C_{12} = M_3 + M_5 \\
&C_{21} = M_2 + M_4 && C_{22} = M_1 - M_2 + M_3 + M_6
\end{aligned}
$$

3 Implementation and practical consideration

3.1 Algorithm schema

We now discuss the algorithm schema for Strassen algorithm for clarity, though results generalize easily for any $\langle m, n, p \rangle$ base case.

The function `cp_matmul` comprises into the following submodules:

1. `dynamic_peeling` is described in Section 3.4.
2. `split_into_blocks` partitions the matrices A and B into subblocks and return them into a flattened array of nodes. For example, at the first iteration of Strassen we might have:

$$
A = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right] \begin{matrix} \} 2^n \\ \\ \end{matrix}
\quad \overbrace{\phantom{A_{11}}}^{2^n} \quad \underbrace{\phantom{A_{21}}}_{2^{n-1}},
$$

   the returned blocks are flattened in row-major order, in this case as $[A_{11}, A_{12}, A_{21}, A_{22}]$.
3. `compute_block_products` Computes the intermediate matrix products $M_r = S_r \cdot T_r$.

3.2 Technical overview

The project burns as a Rust porting of the c++ project made by Benson and Ballard summarized in [2], pubblicaly available in the relative repository under BSD 2-Clause. They provide a fast and efficient framework for benchmarking parallel and sequential fast matrix multiplication algorithms, achieved trough code generation, using lapack and MKL dgemm for numerical computation, and OpenMP for parallelization. Our project differs

Following the work [2] we implement a fast multiplication algorithm based on CP decomposition given the U, V and W matrices representing the algorithm.

We aim to understand how the choice of the base matrix multiplication impacts performance of fast algorithms. To so so, we compare Faer

Fast matrix multiplication algorithm such as Strassen face two significant problems in practical implementation, which are recursion cutoff and input matrices sizes not matching with base case. The former can solved by ...and the latter by dynamic peeling 3.4.

Here comparison with Ballard et Al (used for comparison)

3.3 Recursive cutoff strategies

We implement two strategies for stopping recursion before calling the vendor library, which are by recursion depth and by matrices size. The former stops when a fixed number L of recursion levels is reached, and the latter when matrices sizes reaches a given size.

Cutoff comparisons in sequential case??

3.4 Dynamic Peeling

In Strassen algorithm we want to multiply $\langle m, n, p \rangle$ matrices by splitting in four blocks as in Equation 3. If m, n and p are even, then it is not possible to split blocks in even sizes. The easiest fix for this is padding the matrix with zeros until an even size is reached, but this adds a significant memory waste. A more efficient approach, called *dynamic peeling*, consists in stripping the extra row off and/or column and adding their contribution to the final results. In detail,

For the matrix multiplication $C = AB$, where A is an $m \times n$ matrix and B is an $n \times p$ matrix. The matrix A is split into an $(m-1) \times (n-1)$ core matrix A_{11}, alongside its peeled boundary vectors a_{12}, a_{21}, and the scalar corner element a_{22}. Similarly, the matrix B is split into an $(n-1) \times (p-1)$ core matrix B_{11}, with its corresponding peeled components b_{12}, b_{21}, and b_{22}:

$$A = \left[\begin{array}{c|c} \overbrace{A_{11}}^{n-1} & \overbrace{a_{12}}^{1} \\ \hline a_{21} & a_{22} \end{array}\right] \begin{matrix} \} m-1 \\ \} 1 \end{matrix}, \qquad B = \left[\begin{array}{c|c} \overbrace{B_{11}}^{p-1} & \overbrace{b_{12}}^{1} \\ \hline b_{21} & b_{22} \end{array}\right] \begin{matrix} \} n-1 \\ \} 1 \end{matrix}$$

The final block matrix product is computed directly through a combination of the core Strassen multi-

plication ($A_{11}B_{11}$) and low-rank vector/scalar fixup modifications:

$$\left[\begin{array}{c|c} \overbrace{C_{11}}^{p-1} & \overbrace{c_{12}}^{1} \\ \hline c_{21} & c_{22} \end{array}\right] \begin{array}{l} \} m-1 \\ \} 1 \end{array} = \left[\begin{array}{c|c} A_{11}B_{11} + a_{12}b_{21} & A_{11}b_{12} + a_{12}b_{22} \\ \hline a_{21}B_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{array}\right]$$

Here, only the sub-product $A_{11}B_{11}$ is executed recursively using Strassen's matrix multiplication, while all other terms are computed via standard matrix multiplication.

4 Parallel algorithms for shared memory

4.1 Implementation

We use Rust's Rayon to handle parallelization, and we can summarize the parallel scheduling as following:

- **BFS:** Each recursive matrix multiplication routine and the associated matrix additions are launched by Rayon as an iterable, and then are collected in an array of matrices $[M_1, \ldots, M_r]$.
- **DFS:** Each Faer matrix multiplication call uses all thread, and matrix multiplications are always fully parallelized. implement
- **Hybrid:** TODO.

4.2 Benchmarking

All benchmarks are run with Rust's Criterion library which performs warm-up before doing measurements. In order to compare the performance of matrix multiplication algorithms with different ocmputational costs, we use the *effective* GFLOPS metric for $\langle m, n, r \rangle$ matrix multiplication:

$$\text{effective GFLOPS} = \frac{2mnp - mp}{\text{time in seconds} \cdot 10^9}, \tag{5}$$

where the numerator can be rewritten as $2mnp - mp = mnp + m(n-1)p$, which is the number of multiplications and additions performed for classical matrix multiplication. This allows us to compare all of the algorithms on an inverse-time scale, normalized by problem size [2]. For fast algorithms, considering both multiplications and additions in the metric is more accurate than taking multiplications only [3].

References

[1] Grey Ballard and Tamara G. Kolda. *Tensor Decompositions for Data Science*. Cambridge University Press, 2025.

[2] Austin R. Benson and Grey Ballard. A framework for practical parallel fast matrix multiplication. In *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP 2015, page 42–53, New York, NY, USA, 2015. Association for Computing Machinery.

[3] Benjamin Lipshitz, Grey Ballard, James Demmel, and Oded Schwartz. Communication-avoiding parallel strassen: Implementation and performance. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–11, 2012.

[4] S. Huss-Lederman, E.M. Jacobson, J.R. Johnson, A. Tsao, and T. Turnbull. Implementation of strassen's algorithm for matrix multiplication. In *Supercomputing '96:Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, pages 32–32, 1996.